Tarik Eker
Kilian Hunter
Jun Gi Hong

Lab Course Mobile Roboter
Assignment 6, WS 2024

Group 7
28. Februar 2025

1 Exercise 1 Iterative Closest Point algorithm (ICP)

1.1 a)

We implemented the method ScanMatcher::findMatchesClosest, which identifies the closest point in the reference scan for each point in the new scan. The closest point is determined using the Euclidean distance.

- **Input Transformation:** Each new point is transformed using the provided rotation and translation (represented by the `Transform` structure).
- **Closest Point Search:**
  - For each transformed point, we iterate through all reference points.
  - The Euclidean distance between the transformed point and each reference point is computed.
  - The reference point with the smallest distance is identified as the closest point.
- **Distance Threshold:** If the distance between a new point and its closest reference point exceeds the specified `distanceThreshold`, the match is discarded.
- **Output:** Valid matches are stored as pairs of new points and their corresponding closest reference points.

1.2 b)

Rays which do not hit any obstacle return inf, NaN or values outside of the range of the laser, which are filtered out in the transformLaserScan function.

1.3 c)

In this task, we implemented a dynamic distance threshold for point matching. The distance threshold is defined as:

$$\texttt{distanceThreshold} = \max\left(0.5 \cdot e^{-0.1 \cdot i}, 0.05\right)$$

where i represents the current iteration of the algorithm.
Explanation:

- The exponential decay function ($0.5 \cdot e^{-0.1 \cdot i}$) ensures that the threshold decreases as the algorithm progresses. This allows for tighter matches in later iterations, improving precision. large jumps are prevented.

- The minimum threshold (0.05) prevents the value from becoming too small, ensuring robustness against noise and numerical instabilities.

This approach balances the need for a more relaxed threshold in initial iterations (to handle larger deviations) with higher precision in later iterations.

1.4 d)

We implemented the method `ScanMatcher::findTransformationLSQ`, which computes the transformation minimizing the sum of squared distances between matched points. The steps are as follows:

- **Centroid Calculation:** Compute the centroids of reference (`pRef`) and new (`pNew`) points.
- **Point Centering:** Center the points by subtracting their respective centroids.
- **Covariance Matrix:** Compute components $S_{xx}, S_{xy}, S_{yx}, S_{yy}$ of the cross-covariance matrix.
- **Rotation Angle:** Calculate the rotation angle:

$$\texttt{rotationAngle} = \arctan\left(\frac{S_{xy} - S_{yx}}{S_{xx} + S_{yy}}\right)$$

- **Translation:** Compute the translation:

$$\texttt{translation} = \texttt{centroidRef} - \texttt{centroidNew}.\mathit{rotated}(\texttt{rotationAngle})$$

The method returns the computed transformation as a combination of translation and rotation.

1.5 e)

The Iterative Closest Point (ICP) algorithm is a method for aligning two point clouds by iteratively minimizing the distances between corresponding points. The iterative part involves repeatedly refining the transformation until convergence. The key steps are:

1. **Initial Transformation:** Start with an initial guess for the transformation.
2. **Find Closest Points:** For each point in the new scan, find the closest point in the reference scan.

3. **Compute Transformation:** Calculate the optimal transformation (translation and rotation) that minimizes the distance between matched points.

4. **Apply Transformation:** Transform the new scan using the computed transformation.

5. **Iterate:** Repeat steps 2-4 until the change in error between iterations is below a predefined threshold.

The iterative nature ensures that the alignment improves progressively, starting with coarse matches and refining them with each iteration.

1.6 f)

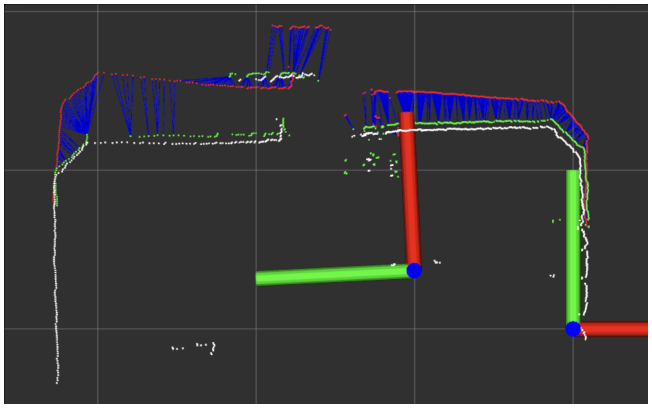

Figure 1: The ICP Algorithm in RVIZ first iteration

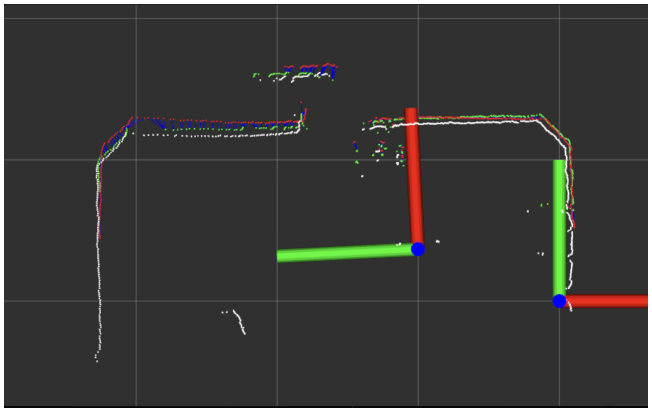

Figure 2: The ICP Algorithm in RVIZ last iteration

2 Exercise 2 Iterative Matching Range Point algorithm (IMRP)

2.1 a)

The difference between the Matching Range Point (MRP) rule and the Closest Point (CP) rule lies in the criteria used for matching points:

- **Closest Point (CP) Rule:** Matches each point in the new scan to the nearest point in the reference scan based on Euclidean distance.
- **Matching Range Point (MRP) Rule:** Matches each point in the new scan to a point in the reference scan that lies within a specified angular range relative to the beam direction.

The MRP rule is not merely a variant of the CP rule but rather introduces an additional constraint (angular range), making it more selective. This is particularly useful in structured environments, as it considers geometric alignment beyond simple proximity.

2.2 b)

We implemented the method `ScanMatcher::findMatchesMRP`, which matches points based on the Matching Range Point (MRP) rule. Below is an overview of the implementation:

- **Point Transformation:** Each new point is transformed using the provided rotation and translation.
- **Matching Rule:** For each transformed point:
  - Compute the vector from the origin to the transformed point and to each reference point.
  - Calculate the angular difference between these vectors and ensure it is within the angular limit B_w.
  - Compare the difference in lengths of the vectors and select the reference point with the smallest length difference, provided it is below 0.15.
- **Validation:** If a valid match is found, it is added to the list of matches.

The MRP rule ensures that matches are constrained not only by proximity but also by angular alignment, making it more robust in structured environments.

Key Constraints:

- Angular difference must be less than B_w.
- Length difference must be below 0.15.

2.3 c)

We implemented the Iterative Matching Range Point (IMRP) algorithm using the provided framework. Below is an overview of the implementation:

- **Initialization:**
  - The initial angular threshold $B_{w_0} = \frac{\pi}{6}$ and translation/rotation thresholds $(0.02, 0.01)$ are defined.
  - A transformation object is initialized to track the cumulative alignment.
- **Iteration Process:** For up to 100 iterations:
  - **Threshold Update:**
    - The angular threshold $B_w = \max(B_{w_0} \cdot e^{-0.05i}, 0.05)$.
    - The distance threshold `distanceThreshold` $= \max(0.5 \cdot e^{-0.1i}, 0.02)$.
  - **Matching:**
    - Matches are computed using both the Closest Point (CP) rule (`findMatchesClosest`) and the Matching Range Point (MRP) rule (`findMatchesMRP`).
  - **Transformation Update:**
    - Transformations are calculated for both sets of matches using Least Squares (`findTransformationLSQ`).
    - The translation and rotation are updated, combining the CP and MRP transformations.
  - **Convergence Check:**
    - The algorithm terminates if the translation and rotation changes are below the thresholds (0.02 and 0.01, respectively).

This approach ensures a robust alignment by combining the strengths of both the Closest Point and Matching Range Point rules.

3 Exercise 3 Iterative Dual Correspondence algorithm (IDC)

We implemented the Iterative Dual Correspondence (IDC) algorithm, which combines the Closest Point (CP) and Matching Range Point (MRP) rules. Below is an overview of the implementation:

- **Dual Matching:**
  - Matches are computed using both:

    - `findMatchesClosest` (Closest Point rule).
    - `findMatchesMRP` (Matching Range Point rule).

- **Dual Transformation:**
  - Transformations are computed independently for each set of matches using the Least Squares method:
    - `transform_1` from CP matches.
    - `transform_2` from MRP matches.
  - The final transformation combines:

    $$\texttt{transform} = (\texttt{transform\_1.t}, \texttt{transform\_2.w})$$

    where the translation from the CP rule and the rotation from the MRP rule are used.

- **Convergence Check:**
  - The algorithm checks whether the change in the differenz between the two last translations (< 0.02) and rotations (< 0.01) falls below the thresholds. If so, it terminates.